

JAVA PROGRAMMING PRACTICAL LAB REPORT

Comprehensive Implementation & Documentation: Methods, Looping, and Conditional Control Structures

Course: Object-Oriented Programming with Java

Document Type: Technical Lab Report

1. SYSTEM REQUIREMENTS

To successfully compile, execute, and evaluate the Java programs detailed in this lab report, the following hardware and software specifications are required:

Component Category	Minimum / Recommended Specification
Operating System	Windows 10/11 (64-bit), macOS 11.0+, or Linux (Ubuntu 20.04 LTS/RedHat)
Processor (CPU)	Intel Core i3 / AMD Ryzen 3 or higher (Multicore processor recommended)
Memory (RAM)	Minimum 4 GB RAM (8 GB or higher recommended for IDE performance)
Storage	500 MB free hard disk space for JDK installation and source files
Java Development Kit (JDK)	Java SE Development Kit 8 / 11 / 17 / 21 LTS (`javac` compiler & `java` runtime)
Development Environment	Command Line Interface (CMD/Terminal) or IDE (VS Code, IntelliJ IDEA, Eclipse)

2. COMPILATION & EXECUTION PROCEDURE

Standard Java program lifecycle involves writing raw code in a `.java` source file, compiling it into bytecode via `javac`, and running it inside the Java Virtual Machine (JVM) via `java`.

Step-by-Step CLI Execution Instructions:

1. Save Source Files:

Create files corresponding to class names:

- MethodTask.java
- LoopState.java
- ConditionalState.java

2. Open Terminal/Command Prompt:

Navigate to the target directory containing the source files:

```
cd /path/to/java/source/files
```

3. Compile Source Code:

Execute the Java compiler command:

```
javac MethodTask.java LoopState.java ConditionalState.java
```

(This produces bytecode .class files if there are no syntax errors).

4. Execute Compiled Programs:

Run Method program: `java MethodTask`

Run Loop program: `java LoopState`

Run Conditional program: `java ConditionalState`

3. MODULE 1: JAVA METHODS (METHODTASK)

Overview & Objective:

Demonstrate modular programming in Java using static methods. This module contains 4 utility tasks: calculating factorials, verifying prime numbers, finding the maximum of two numbers, and computing the area of a circle.

Task 1.1: Factorial Calculation

Objective: Compute the factorial of a user-defined non-negative integer using an iterative static method.

Algorithm:

1. Start program and initialize Scanner for console input.
2. Read integer n from user.
3. Call factorial(n) method:
 - Initialize fact = 1.
 - Loop i from 1 up to n: multiply fact = fact * i.
 - Return fact.
4. Print the returned factorial value.

UML Class Diagram & Flowchart

```
+-----+
|           MethodTask           |
+-----+
| + static factorial(n: int): int |
| + main(args: String[]): void  |
+-----+
```

[Start] -> [Read n] -> [Call factorial(n)] -> [fact = 1] -> [Loop i=1..n: fact*=i] -> [Return fact] -> [Display R

Source Code

```
import java.util.Scanner;

public class MethodTask {
    static int factorial(int n) {
        int fact = 1;
        for (int i = 1; i <= n; i++) {
            fact = fact * i;
        }
        return fact;
    }

    public static void main(String[] args) {
        Scanner fa = new Scanner(System.in);
        int n, result;
        System.out.println("enter the number");
        n = fa.nextInt();
        result = factorial(n);
        System.out.println("factorial = " + result);
    }
}
```

```
}  
}
```

Execution Output

```
enter the number  
5  
factorial = 120
```

Task 1.2: Prime Number Verification

Objective: Determine whether an integer input is a prime number using a custom boolean method.

Algorithm:

1. Accept input integer n.
2. In isprime(n):
 - If $n \leq 1$, return false.
 - Loop i from 2 to $n/2$. If $n \% i == 0$, return false.
 - If no divisor found, return true.
3. If isprime(n) is true, print "PRIME NUMBER", else print "not a prime number".

UML Flowchart

```
[Start] -> [Read n] -> [n <= 1?] --Yes--> [Return false]  
                        | No  
                        [i=2 to n/2]  
                        [n % i == 0?] --Yes--> [Return false]  
                        | No  
                        [Return true] -> [Display PRIME / NOT PRIME] -> [End]
```

Source Code

```
import java.util.Scanner;  
  
public class MethodTask {  
    static boolean isprime(int n) {  
        if (n <= 1)  
            return false;  
        for (int i = 2; i <= n / 2; i++) {  
            if (n % i == 0)  
                return false;  
        }  
        return true;  
    }  
  
    public static void main(String[] args) {  
        Scanner pr = new Scanner(System.in);  
        int n;  
        System.out.println("enter the nnumber");  
        n = pr.nextInt();  
        if (isprime(n))  
            System.out.println("PRIME NUMBER");  
        else  
            System.out.println("not a prime number");  
    }  
}
```

```
}  
}
```

Execution Output

```
enter the nnumber  
17  
PRIME NUMBER
```

Task 1.3: Maximum of Two Numbers

Objective: Find the maximum between two integer inputs using conditional method logic.

Algorithm:

1. Read two integers a and b.
2. Execute `max(a, b)`: Return a if `a > b`, else return b.
3. Output the returned maximum value.

Source Code

```
import java.util.Scanner;

public class MethodTask {
    static int max(int a, int b) {
        if (a > b)
            return a;
        else
            return b;
    }

    public static void main(String[] args) {
        Scanner mt = new Scanner(System.in);
        int a, b, maxVal;
        System.out.println("enter the values of a,b");
        a = mt.nextInt();
        b = mt.nextInt();
        maxVal = max(a, b);
        System.out.println("maximum is " + maxVal);
    }
}
```

Execution Output

```
enter the values of a,b
14 29
maximum is 29
```

Task 1.4: Area of Circle

Objective: Compute the geometric area of a circle given its floating-point radius.

Algorithm:

1. Read double precision float radius.
2. Compute area using method formula: `3.14 * radius * radius`.
3. Return and display calculated area.

Source Code

```
import java.util.Scanner;
```

```

public class MethodTask {
    static double area(double radius) {
        return 3.14 * radius * radius;
    }

    public static void main(String[] args) {
        Scanner r = new Scanner(System.in);
        double radius, result;
        System.out.println("enter the radius");
        radius = r.nextDouble();
        result = area(radius);
        System.out.println("area = " + result);
    }
}

```

Execution Output

```

enter the radius
7.0
area = 153.86

```

4. MODULE 2: LOOPING CONTROL STRUCTURES (LOOPSTATE)

Overview & Objective:

Implement repetition logic using for loops. This module covers multiplication table generation, range displaying, natural number summation, and Fibonacci sequence generation.

Task 2.1: Multiplication Table

Objective: Print the multiplication table of a given number from 1 to 10.

Algorithm: Read integer n. Iterate i from 1 to 10 and print $n \times i = (n*i)$.

Source Code & Execution Output

```

import java.util.Scanner;

public class LoopState {
    public static void main(String[] args) {
        Scanner m = new Scanner(System.in);
        int n;
        System.out.println("enter the number");
        n = m.nextInt();
        System.out.println("the multipilcation table of given number");
        for (int i = 1; i <= 10; i++) {
            System.out.println(n + "x" + i + " = " + (n * i));
        }
    }
}

```

```

enter the number
6
the multipilcation table of given number
6x1 = 6

```

```
6x2 = 12
6x3 = 18
6x4 = 24
6x5 = 30
6x6 = 36
6x7 = 42
6x8 = 48
6x9 = 54
6x10 = 60
```

Task 2.2: Numbers Display (1 to n)

Objective: Iterate and print sequential numbers up to boundary n.

Algorithm: Read integer n. Loop i from 1 to n and print i.

Source Code & Output

```
import java.util.Scanner;

public class LoopState {
    public static void main(String[] args) {
        Scanner N = new Scanner(System.in);
        int n;
        System.out.println(" enter the number");
        n = N.nextInt();
        System.out.println("Numbers from 1 to n " + (n));
        for (int i = 1; i <= n; i++) {
            System.out.println(i);
        }
    }
}
```

```
enter the number
5
Numbers from 1 to n 5
1
2
3
4
5
```

Task 2.3: Sum of Natural Numbers

Objective: Accumulate the total sum of numbers from 1 up to n.

Algorithm: Initialize sum = 0. Loop i from 1 to n and set sum = sum + i. Output sum.

Source Code & Output

```
import java.util.Scanner;

public class LoopState {
    public static void main(String[] args) {
        Scanner sn = new Scanner(System.in);
        int n;
        System.out.println("enter the number");
        n = sn.nextInt();
        int sum = 0;
        for (int i = 1; i <= n; i++) {
            sum = sum + i;
        }
        System.out.println("sum = " + sum);
    }
}
```

```
}
}
```

```
enter the number
10
sum = 55
```

Task 2.4: Fibonacci Series

Objective: Generate the first n terms of the Fibonacci sequence.

Algorithm: Initialize $a = 0$, $b = 1$. Loop i from 1 to n : print a , compute $c = a + b$, set $a = b$, $b = c$.

UML Flowchart Diagram

```
+-----+
|                               |
|      LoopState (Fibonacci)   |
|                               |
| + main(args: String[]): void |
|                               |
+-----+

[Start] -> [Read n] -> [a=0, b=1, i=1] -> [Print a] -> [c = a+b] -> [a=b, b=c] -> [i++] -> [i <= n?] --Yes--> [Loop]
                                     | No
                                     +-----> [End]
```

Source Code & Output

```
import java.util.Scanner;

public class LoopState {
    public static void main(String[] args) {
        Scanner f = new Scanner(System.in);
        int n;
        System.out.println("enter n");
        n = f.nextInt();
        int a = 0;
        int b = 1;
        System.out.println("the fibonacci series");
        for (int i = 1; i <= n; i++) {
            System.out.print(a + " ");
            int c = a + b;
            a = b;
            b = c;
        }
        System.out.println();
    }
}
```

```
enter n
7
the fibonacci series
0 1 1 2 3 5 8
```

5. MODULE 3: CONDITIONAL CONTROL STATEMENTS (CONDITIONAL STATE)

Overview & Objective:

Apply decision-making constructs using if-else, else-if ladders, and switch-case blocks.

Task 3.1: Even or Odd Verification

Objective: Check parity of integer n using modulus operator % 2.

Source Code & Output

```
import java.util.Scanner;

public class ConditionalState {
    public static void main(String[] args) {
        Scanner eo = new Scanner(System.in);
        int n;
        System.out.println("enter the number");
        n = eo.nextInt();
        if (n % 2 == 0) {
            System.out.println("the number is even");
        } else {
            System.out.println("the number is odd");
        }
    }
}
```

```
enter the number
24
the number is even
```

Task 3.2: Largest Among Three Numbers

Objective: Find the maximum of three integers using compound logical conditions (&&).

Source Code & Output

```
import java.util.Scanner;

public class ConditionalState {
    public static void main(String[] args) {
        Scanner max = new Scanner(System.in);
        int a, b, c;
        System.out.println("enter the values of a,b,c");
        a = max.nextInt();
        b = max.nextInt();
        c = max.nextInt();
        if (a >= b && a >= c) {
            System.out.println("the largest number is " + a);
        } else if (b >= a && b >= c) {
            System.out.println("the largest number is " + b);
        }
    }
}
```

```

    } else {
        System.out.println("the largest number is " + c);
    }
}
}

```

```

enter the values of a,b,c
45 89 12
the largest number is 89

```

Task 3.3: Grade Percentage Calculation

Objective: Classify academic performance grades using an else-if ladder based on percentage thresholds.

Source Code & Output

```

import java.util.Scanner;

public class ConditionalState {
    public static void main(String[] args) {
        Scanner p = new Scanner(System.in);
        double percentage;
        System.out.println("enter the percentage");
        percentage = p.nextDouble();
        if (percentage >= 90) {
            System.out.println("GRADE - A");
        } else if (percentage >= 80) {
            System.out.println("GRADE - B");
        } else if (percentage >= 70) {
            System.out.println("GRADE - C");
        } else if (percentage >= 60) {
            System.out.println("GRADE - D");
        } else {
            System.out.println("FAIL");
        }
    }
}

```

```

enter the percentage
84.5
GRADE - B

```

Task 3.4: Day of the Week Display

Objective: Map an integer index (1-6) to weekdays using a multi-branch switch-case statement.

Source Code & Output

```

import java.util.Scanner;

public class ConditionalState {
    public static void main(String[] args) {
        Scanner d = new Scanner(System.in);
        int day;
        System.out.println("enter the day");
    }
}

```

```
    day = d.nextInt();

    switch (day) {
        case 1:
            System.out.println("sunday");
            break;
        case 2:
            System.out.println("monday");
            break;
        case 3:
            System.out.println("tuesday");
            break;
        case 4:
            System.out.println("wednesday");
            break;
        case 5:
            System.out.println("thursday");
            break;
        case 6:
            System.out.println("friday");
            break;
        default:
            System.out.println("invalid day");
            break;
    }
}
```

```
enter the day
3
tuesday
```